DOJOON · POS LINK

Android SDK Developer Guide

Connect a cashier app to a Sunmi P3 payment terminal over USB or Bluetooth, and take card payments safely.

Version	0.1.0
Package	`ly.dojoon.poslink:poslink-android:0.1.0`
Requires	Android 7.0 (API 24)+, compileSdk 34+, Kotlin 2.0+
Source	github.com/Alharudi/poslink-sdk
Date	12 November 2026

Apache License 2.0

Contents

1 **Introduction** 4

Start here 4
How it works 4
Platforms 5

2 **Android setup** 7

Requirements 7
1. Add the dependency 7
2. Permissions 8
3. USB: let Android open your app when the cable is plugged in 8
4. Bluetooth: pair the devices in Android settings once 9
5. Pick a transport 10
Keep one instance for the whole app 10
Files the SDK keeps 10
Code shrinking (R8) 11

3 **Cashier guide (Android)** 12

1. Create and start 12
2. Show the connection status 12
3. Pair with the terminal (once) 13
4. Take a payment 14
5. Handle the outcome 15
6. When the outcome is Unknown 16
7. Cancel a payment 16
Threads 16
Reference 17

4 **Terminal guide (Android)** 18

1. Create and start 18
2. Show the payment on screen 18
3. Pair a cashier 19
4. Connect your card payment (PaymentAdapter) 20
5. What the SDK enforces for you 21
Threads 22
Known limitations 22
Reference 23

5 **Testing without devices or cards** 24

Unit tests (JVM) 24
Two phones on the same Wi-Fi 26
The sample app 26

6 Payment safety checklist 27
The three rules 27
Cashier app 27
Terminal app 27
Failure drills 28
What the link protects against 28
7 Troubleshooting 29
The status stays OFFLINE 29
Stuck at HANDSHAKING 30
UNPAIRED, although it worked before 30
Stuck at AUTHENTICATING, or "failed the security check" 30
Pairing fails 30
Sales 31
Bluetooth LE 31
Installing on Sunmi devices 31

CHAPTER 1

Introduction

POS Link connects a **cashier app** (the till) to a **payment terminal app** (on a Sunmi P3) over a USB cable or Bluetooth. The cashier asks for a payment, the customer pays on the terminal, and the result comes back to the cashier.

Start here

I want to...	Read
Understand how it works (5 minutes)	How it works, below
Add it to an Android app	Android setup
Write the cashier side (take payments)	Cashier guide
Write the terminal side (the app on the P3)	Terminal guide
Test without devices or cards	Testing
Go live without double charges	Payment safety checklist
Fix a problem	Troubleshooting
Build an SDK for another language	Protocol spec and test vectors
Publish a new version (maintainers)	Releasing

How it works

Two roles

- **Cashier:** the app where the order is made (a till, a tablet, a PC). It *connects*.
- **Terminal:** the app on the payment device that reads the card. It *waits* for the cashier.

One cashier talks to one terminal at a time.

Three ways to connect

Link	Cashier	Terminal	Notes
USB cable	USB host (`UsbHostTransport`)	USB accessory (`UsbAccessoryTransport`)	Most reliable. The terminal needs no developer options.
Bluetooth	`BluetoothClientTransport`	`BluetoothServerTransport`	Bluetooth Classic. Pair the two devices in Android settings first.
Wi-Fi / LAN	`TcpClientTransport`	`TcpServerTransport`	For development and fixed LAN setups.

Bluetooth LE is not used: Sunmi terminals in payment-security (PCI) mode switch it off.

Pair once, then every message is signed

The first time a cashier meets a terminal, they **pair**:

1. The merchant taps **Pair a cashier** on the terminal (pairing stays open for 2 minutes).
2. The cashier starts pairing. The terminal shows a **6-digit code**.
3. The person types that code on the cashier, then taps **Confirm** on the terminal.

Both sides now share a secret key. After that, every message between them is signed with it, so a device in the middle can't fake an "approved" or replay an old one. Pairing survives restarts. Pair again only if one app is reinstalled or its data is cleared.

This is separate from Bluetooth pairing in Android settings, which is also needed for Bluetooth.

A sale has three possible outcomes

```
when (val outcome = cashier.sale(amount = 12500, currency = "LYD")) {
    is SaleOutcome.Completed -> // the terminal's final answer: approved, declined, cancelled…
    is SaleOutcome.Refused   -> // refused before any card was read: no money moved
    is SaleOutcome.Unknown   -> // nobody knows yet whether money moved: NEVER treat as declined
}
```

`Unknown` happens when the link drops in the middle of a payment. The SDK keeps asking the terminal and tells you the real result when the link is back. See the payment safety checklist.

Built-in protection

- **No double charges:** each sale has a `txnId`. The terminal charges a `txnId` at most once; a repeated request gets the stored result back.
- **One sale at a time:** a second sale while one is running is refused (`BUSY`).
- **Lost messages:** every message is acknowledged and retried.
- **Pulled cable or app restart:** results are kept and re-sent for 24 hours until the cashier confirms it got them.

Amounts

Amounts are whole numbers in the currency's smallest unit. Libyan dinar has 3 decimals, so 12.500 LYD is `12500`. Currency is the 3-letter ISO 4217 code (`"LYD"`).

Platforms

Cashier runs on	Status
Android	✅ Android SDK (this repository)
Flutter	Planned (wraps the Android SDK)
Windows / macOS / Linux, web (Chrome/Edge)	Planned (TypeScript and .NET SDKs)
iPhone / iPad	✕ Apple doesn't allow it without MFi hardware

The terminal side always runs on Android (the Sunmi P3) and uses the Android SDK.

Until an SDK exists for your platform, you can implement the protocol yourself. The test vectors tell you when your implementation is byte-for-byte right.

CHAPTER 2

Android setup

The same library is used on both sides: the cashier app and the terminal app.

Requirements

- Runs on Android 7.0 (API 24) or newer.
- Your app compiles against Android SDK 34 or newer (`compileSdk = 34`).
- Kotlin 2.0 or newer. Kotlin coroutines (1.9.0 or newer) come with the library.
- Java callers work too, but the API is built for Kotlin (`suspend` functions and `StateFlow`).

1. Add the dependency

The library is hosted on the project's GitHub Pages. Add that address once, in `settings.gradle.kts`:

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositories {
        google()
        mavenCentral()
        maven {
            url = uri("https://alharudi.github.io/poslink-sdk/maven")
            content { includeGroup("ly.dojoon.poslink") } // only POS Link comes from here
        }
    }
}
```

Then add the library to your app:

```
// app/build.gradle.kts
dependencies {
    implementation("ly.dojoon.poslink:poslink-android:0.1.0")
}
```

The library will also go to Maven Central later. From then on, the `maven { … }` block isn't needed; the dependency line stays the same.

> **Got the SDK as a zip file instead?**
>
> Unzip it (for example to `libs/poslink-repo` in your project), then add that folder as a repository in `settings.gradle.kts`, instead of the web address:
>
> ```
> dependencyResolutionManagement {
>     repositories {
>         google()
>         mavenCentral()
>         maven { url = uri("libs/poslink-repo") }
>     }
> }
> ```
>
> The dependency line stays the same.

2. Permissions

The library's manifest already adds these to your app:

Permission	Why
`BLUETOOTH_CONNECT` (Android 12+)	Connect over Bluetooth
`BLUETOOTH_ADVERTISE` (Android 12+)	Terminal: be visible for Bluetooth pairing
`BLUETOOTH`, `BLUETOOTH_ADMIN` (Android 11 and older)	Same, older Android

USB needs no permission in the manifest. Android asks the user once with a system dialog.

On **Android 12 and newer**, the Bluetooth permissions must also be granted at runtime ("Nearby devices"). The SDK can't show the prompt, so your app asks before or after `start()`:

```
val needed = BluetoothClientTransport.requiredPermissions() // or BluetoothServerTransport's
val missing = needed.filter { checkSelfPermission(it) != PackageManager.PERMISSION_GRANTED }
if (missing.isNotEmpty()) requestPermissions(missing.toTypedArray(), REQUEST_BLUETOOTH)
```

Until they are granted, the link reports `TransportStatus.PERMISSION` and waits. It connects on its own once they are granted. Nothing needs restarting.

Wi-Fi link only: add `<uses-permission android:name="android.permission.INTERNET" />` yourself. The library doesn't add it.

3. USB: let Android open your app when the cable is plugged in

This step is optional but recommended. Without it, USB still works, but the user gets a permission dialog each time.

Add the intent filter for your side to your main activity. The XML filter files come with the library:

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:launchMode="singleTop">

    <!-- Cashier app: a terminal was plugged in -->
    <intent-filter>
        <action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
    </intent-filter>
    <meta-data
        android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
        android:resource="@xml/poslink_device_filter" />

    <!-- Terminal app: a cashier connected -->
    <intent-filter>
        <action android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED" />
    </intent-filter>
    <meta-data
        android:name="android.hardware.usb.action.USB_ACCESSORY_ATTACHED"
        android:resource="@xml/poslink_accessory_filter" />
</activity>
```

Then pass the activity's intents to the transport:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    usbTransport.handleIntent(intent)
}

override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    usbTransport.handleIntent(intent)
}
```

If the user ticks **"Always open"** in the dialog, Android remembers the choice and stops asking.

> **Your terminal isn't a Sunmi P3?** `poslink_device_filter` lists the P3's USB id (`0E8D:201C`). For other devices, copy the file into your app's `res/xml/` under a new name, add your device's vendor and product id (in decimal), and point the `meta-data` at your copy. Always match vendor **and** product id, because many devices share a vendor id.

4. Bluetooth: pair the devices in Android settings once

Bluetooth needs the two devices paired in Android settings first. This is separate from POS Link pairing.

1. On the terminal, open **Settings → Bluetooth**. It is visible to others while that screen is open.
2. On the cashier, open **Settings → Bluetooth**, tap the terminal's name and confirm the code on both devices.

Then let the cashier user pick the terminal from the paired devices:

```
val devices = PosLinkBluetooth.bondedDevices(context) // null until BLUETOOTH_CONNECT is granted
// show devices (name + address); save the chosen address
val transport = BluetoothClientTransport(context, chosen.address)
```

`hasPosLinkService` tells you which devices have offered the POS Link service before. Use it to sort the list, not to hide devices: Android's cached list can be out of date.

5. Pick a transport

	Cashier app	Terminal app
USB	`UsbHostTransport(context)`	`UsbAccessoryTransport(context)`
Bluetooth	`BluetoothClientTransport(context, address)`	`BluetoothServerTransport(context)`
Wi-Fi	`TcpClientTransport(host, port = 8765)`	`TcpServerTransport(port = 8765)`

A transport reconnects on its own: after a pulled cable, Bluetooth out of range, or a terminal restart. You never call connect again.

Next: the cashier guide or the terminal guide.

Keep one instance for the whole app

Create the `PosLinkCashier` or `PosLinkTerminal` once, for example in your `Application` class or a singleton, and call `start()` once. Don't create one per screen: an Activity is destroyed on every rotation, while the link must survive it.

```
class App : Application() {
    lateinit var cashier: PosLinkCashier
        private set

    override fun onCreate() {
        super.onCreate()
        cashier = PosLinkCashier(/* see the cashier guide */)
        cashier.start()
    }
}
```

Files the SDK keeps

What	Where	If deleted
This app's POS Link id	SharedPreferences `poslink_identity`	The app looks like a new device; pair again
Pairing keys (`KeystorePairingStore`)	`noBackupFilesDir/poslink_pairings.json`, encrypted with a key in the Android Keystore	Pair again

What	Where	If deleted
Transactions (`FileJournal`)	The file you pass in	The duplicate-charge and recovery history is lost
USB terminals picked on the cashier	SharedPreferences `poslink_usb`	Pick the device again

Pairing keys are in `noBackupFilesDir`, so they are never copied by Android backup. The Keystore key can't leave the device, so a copied file is useless anywhere else.

Code shrinking (R8)

Nothing to configure. Release builds with R8 turned on work as they are.

CHAPTER 3

Cashier guide (Android)

The cashier app asks the terminal for payments. Do the setup first.

- 1. Create and start
- 2. Show the connection status
- 3. Pair with the terminal (once)
- 4. Take a payment
- 5. Handle the outcome
- 6. When the outcome is Unknown
- 7. Cancel a payment
- Threads · Reference

1. Create and start

```
val cashier = PosLinkCashier(
    transport = BluetoothClientTransport(context, terminalAddress), // or UsbHostTransport(context)
    identity = AndroidIdentity.load(context, name = "Front desk"),   // the name the terminal shows
    pairings = KeystorePairingStore(context),
    journal = FileJournal(File(context.filesDir, "poslink-cashier.json")),
    info = AndroidIdentity.deviceInfo(appVersion = BuildConfig.VERSION_NAME),
    log = { Log.d("PosLink", it) },
)
cashier.start()
```

Create it **once** for the whole app (see setup). `start()` returns at once. The link connects in the background and reconnects on its own. Call `stop()` (a `suspend` function) only when the app no longer needs the terminal.

2. Show the connection status

`cashier.status` is a `StateFlow<LinkStatus>`:

Status	Meaning	Show the user
`OFFLINE`	No cable, Bluetooth off or out of range, terminal app closed	"Terminal not connected"
`HANDSHAKING`, `AUTHENTICATING`	Connected, checking each other	"Connecting…"
`UNPAIRED`	The terminal answered, but the two aren't paired	A **Pair** button (step 3)
`PAIRING`	A pairing is running	"Pairing…"
`ONLINE`	Ready for payments	"Terminal ready"
`NOT_RESPONDING`	Was online, the terminal stopped answering for 10 s	"Terminal not responding"

```
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        cashier.status.collect { status ->
            payButton.isEnabled = status == LinkStatus.ONLINE
            pairButton.isVisible = status == LinkStatus.UNPAIRED
            statusText.text = describe(status)
        }
    }
}
```

More detail, when you need it:

Property	What it tells you
`cashier.link.transportState`	The raw connection, e.g. `PERMISSION` (ask for Bluetooth permissions), `ERROR: Bluetooth is off…`. `detail` is a sentence you can show.
`cashier.link.problem`	Something that blocks the link and that the user must fix, e.g. both devices are set up as cashiers. `null` when all is well.
`cashier.link.peer`	The terminal: `name`, `deviceId`, `appVersion`.
`cashier.terminalState`	`"idle"` or `"busy"`, as the terminal last reported.

3. Pair with the terminal (once)

A person does two things: opens pairing on the terminal, then types the terminal's code on the cashier.

```
// When the user taps "Pair" (status is UNPAIRED):
val result = cashier.pair { code ->
    // Runs on the SDK's thread: switch to the main thread for UI.
    val typed = withContext(Dispatchers.Main) {
        askUserToType("Enter the 6-digit code shown on the terminal")
    }
    typed == code
}
when (result) {
    is PairingResult.Paired -> showMessage("Paired with ${result.peerName}")
    is PairingResult.Failed -> showMessage(result.message)
}
```

Tell the user to tap **Pair a cashier** on the terminal *before* tapping Pair here. The terminal then shows the code and a Confirm button.

Make the user type the code. Don't show both codes with a "Do they match?" button: people tap yes without looking. Typing is what stops a device in the middle.

Your `confirm` block is cancelled if the terminal gives up first (rejected there, or after 2 minutes), so close your dialog when the coroutine is cancelled.

`result.reason`	Why	What to do
`NOT_PAIRING`	Pairing isn't open on the terminal	Tap **Pair a cashier** on the terminal, then try again

`result.reason`	Why	What to do
`REJECTED`	Someone tapped Reject, or `confirm` returned `false`	Try again
`BAD_CONFIRM`	The codes didn't match	Try again; check you're near the right terminal
`TIMEOUT`	Nobody finished within 2 minutes	Try again
`BUSY`	The terminal is pairing with another cashier	Wait, then try again
`NOT_CONNECTED`	No terminal on the link	Check the cable or Bluetooth
`BAD_KEY`, `BAD_COMMIT`	The terminal's answer was invalid	Try again. If it repeats, something is tampering with the link

The pairing is saved on both devices and survives restarts. `cashier.pairedTerminals()` lists the saved pairings, and `cashier.forget(terminalDeviceId)` deletes one.

Pair again only when status shows `UNPAIRED` after it worked before. That means one of the two apps was reinstalled or had its data cleared.

4. Take a payment

```
// Save the txnId with the order BEFORE calling sale(), so you can always ask about it later.
val txnId = order.paymentTxnId ?: UUID.randomUUID().toString().also { saveTxnId(order, it) }

val outcome = cashier.sale(
    amount = 12500,           // 12.500 LYD, in minor units
    currency = "LYD",
    reference = order.number, // optional; shown in logs and reports
    txnId = txnId,
) { event ->
    // Runs on the SDK's thread.
    runOnUiThread { showProgress(event) }
}
```

`sale()` suspends until the payment ends, usually 10 to 60 seconds. Call it from a coroutine (for example `lifecycleScope.launch { … }`). Only one sale runs at a time per cashier.

Progress events, in order:

`SaleEvent`	Meaning
`Sent(txnId)`	The request is on its way
`ReadyToPay`	The terminal shows the amount and waits for the card
`Progress(stage)`	`CARD_READ`, `PIN_ENTRY`, `AUTHORIZING`, `PRINTING`
`ConnectionLost`	The link dropped mid-payment; the sale keeps waiting
`Reconnected`	The link is back; the SDK checks the sale's status

Cancelling the coroutine that called `sale()` only stops *waiting*. The payment goes on at the terminal. To cancel the payment itself, see step 7.

5. Handle the outcome

```
when (outcome) {
    is SaleOutcome.Completed -> {
        val result = outcome.result
        if (result.approved) markPaid(order, result)         // print the receipt
        else showNotPaid(result.outcome, result.message)     // declined, cancelled, timeout, error
    }
    is SaleOutcome.Refused -> showError(outcome.message)     // no money moved: safe to try again
    is SaleOutcome.Unknown -> markNeedsCheck(order, outcome.message) // see step 6
}
```

Outcome	Money taken?	What to do
`Completed` with `result.approved`	**Yes**	Mark paid, print the receipt
`Completed`, not approved (`DECLINED`, `CANCELLED`, `TIMEOUT`, `ERROR`)	No	Show `result.message`; the customer may pay another way
`Refused`	No	Show `message`; trying again is safe
`Unknown`	**Maybe**	Don't charge again with a new `txnId`. See step 6

`Refused.code` is `BUSY` (terminal busy with another payment), `BAD_REQUEST` (invalid amount, currency or `txnId`), or `null` (the terminal is not connected, or it never got the sale).

What the result carries (`SaleResult`):

Field	Example	Notes
`outcome`, `approved`	`APPROVED`, `true`	
`amount`, `currency`	`12500`, `"LYD"`	What was actually charged
`responseCode`, `message`	`"00"`, `"Approved"`	From the bank
`authCode`, `rrn`, `stan`	`"123456"`, …	For the receipt and for disputes
`maskedPan`, `cardScheme`	`"476173******0119"`, `"VISA"`	Never the full card number
`terminalId`, `merchantId`		
`timestamp`	epoch milliseconds	

Any field except `outcome`, `amount` and `currency` can be `null`, depending on the bank and on how far the payment got.

6. When the outcome is Unknown

`Unknown` means the link dropped (or the terminal went silent for 150 s) during the payment. The card **may have been charged**. Never treat it as declined.

You don't have to poll. When the link comes back, the SDK asks the terminal about every unfinished sale from the last 24 hours and reports each answer on `cashier.updates`. Collect it once, app-wide, right after creating the cashier (it doesn't replay past changes):

```
appScope.launch {
    cashier.updates.collect { record ->
        if (record.state == TxnState.COMPLETED) applyResult(record.txnId, record.result!!)
    }
}
```

To ask now, for example from a "Check payment" button:

```
val status = cashier.status(txnId) // null if the terminal didn't answer within 10 s
when (status?.state) {
    TxnStatusState.COMPLETED -> applyResult(txnId, status.result!!)
    TxnStatusState.IN_PROGRESS -> showMessage("Still paying on the terminal")
    TxnStatusState.NOT_FOUND -> showMessage("The terminal never got it. No money was taken.")
    TxnStatusState.INTERRUPTED -> showMessage("The terminal restarted during the payment. Check its
bank report.")
    null -> showMessage("Can't reach the terminal")
}
```

After an app restart, `cashier.transactions()` still lists the sales whose state is `UNKNOWN`, so you can show them as "needs checking".

You can also simply call `sale()` again **with the same `txnId`**. The terminal charges a `txnId` at most once: if it already finished, you get the stored result back and nothing is charged. A **new** `txnId` is a new charge.

7. Cancel a payment

```
cashier.cancel(txnId) // true when the terminal got the request
```

This works only before the customer presents a card. If it worked, the running `sale()` ends with `Completed` and outcome `CANCELLED`. If the card was already read, nothing changes: the payment finishes normally.

Threads

What	Runs on	Note
`sale`, `pair`, `status`, `cancel`, `stop`	Any coroutine (main is fine)	They suspend; they never block
The `onEvent` callback of `sale`	The SDK's thread	Switch to main for UI
The `confirm` block of `pair`	The SDK's thread	Use `withContext(Dispatchers.Main)` for UI
`StateFlow`s and `updates`	Wherever you collect them	

Reference

```
class PosLinkCashier(
    transport: Transport,
    identity: DeviceIdentity,
    pairings: PairingStore,
    journal: TransactionJournal,
    config: LinkConfig = LinkConfig(),          // timings; the defaults are tuned for USB and
Bluetooth
    info: Map<String, Any?> = emptyMap(),       // shown to the terminal:
AndroidIdentity.deviceInfo()
    noUpdateTimeoutMs: Long = 150_000,          // silence during a sale before it becomes Unknown
    log: (String) -> Unit = {},
    // dispatcher, clock, scope: for tests
)

fun start()
suspend fun stop()

val status: StateFlow<LinkStatus>
val terminalState: StateFlow<String?>          // "idle" / "busy"
val updates: SharedFlow<TxnRecord>             // every journal change, incl. recovered results
val link: LinkSession                          // transportState, problem, peer, stats

suspend fun pair(confirm: suspend (code: String) -> Boolean): PairingResult
fun pairedTerminals(): List<Pairing>
fun forget(terminalDeviceId: String)

suspend fun sale(amount: Long, currency: String, reference: String? = null,
                 txnId: String = UUID.randomUUID().toString(),
                 onEvent: (SaleEvent) -> Unit = {}): SaleOutcome
suspend fun cancel(txnId: String): Boolean
suspend fun status(txnId: String, timeoutMs: Long = 10_000): TxnStatus?
fun transactions(): List<TxnRecord>            // newest first, from the journal
```

The generated API reference (KDoc) is attached to the library, so Android Studio shows it on hover.

CHAPTER 4

Terminal guide (Android)

The terminal app runs on the payment device (a Sunmi P3). It waits for a cashier, runs each sale through your card payment code, and sends the result back. Do the setup first.

- 1. Create and start
- 2. Show the payment on screen
- 3. Pair a cashier
- 4. Connect your card payment (PaymentAdapter)
- 5. What the SDK enforces for you
- Threads · Known limitations · Reference

1. Create and start

```
val terminal = PosLinkTerminal(
    transport = UsbAccessoryTransport(context), // or BluetoothServerTransport(context)
    identity = AndroidIdentity.load(context, name = "Counter 1"), // the name the cashier shows
    pairings = KeystorePairingStore(context),
    journal = FileJournal(File(context.filesDir, "poslink-terminal.json")),
    payment = BankPayment(/* … */),             // your adapter, see step 4; SimulatedPayment() for
demos
    info = AndroidIdentity.deviceInfo(appVersion = BuildConfig.VERSION_NAME),
    log = { Log.d("PosLink", it) },
)
terminal.start()
```

Create it **once** for the whole app and keep it running while the app is open. One terminal serves one link: let the merchant choose USB or Bluetooth in your settings.

`terminal.status` gives the same `LinkStatus` values as on the cashier (table). `ONLINE` means a paired cashier is connected.

2. Show the payment on screen

`terminal.phase` says what the screen should show:

`TerminalPhase`	Show
`Idle`	Your home screen ("Ready")
`WaitingCard(request)`	The amount (`request.amount` , `request.currency`) and "Present card", plus a Cancel button
`Processing(request, stage)`	"Reading card…", "Enter PIN…", "Authorizing…", "Printing…"
`Result(result)`	Approved or declined, with `result.message`

```
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        terminal.phase.collect { phase -> render(phase) }
    }
}
```

- The result stays for 6 seconds, then the phase goes back to `Idle`. Call `terminal.dismissResult()` to go back sooner.
- A Cancel button calls `terminal.cancelOnTerminal()`. It works only before a card is presented, and the cashier then gets outcome `CANCELLED`.

If your bank's payment app takes over the screen during the payment, you may only need `Idle` and `Result`.

3. Pair a cashier

Pairing is done once per cashier. It starts here, on the terminal.

```
// "Pair a cashier" button in your settings:
pairButton.setOnClickListener { terminal.openPairing() } // open for 2 minutes

lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        terminal.pairingRequest.collect { request ->
            if (request == null) {
                closePairingDialog()
            } else {
                showPairingDialog(
                    title = "Pair with ${request.cashierName}?",
                    code = request.code,                // show it big: the cashier types it
                    onConfirm = { request.confirm() },
                    onReject = { request.reject() },
                )
            }
        }
    }
}

lifecycleScope.launch {
    terminal.pairingResults.collect { result ->
        when (result) {
            is PairingResult.Paired -> showMessage("Paired with ${result.peerName}")
            is PairingResult.Failed -> showMessage(result.message)
        }
    }
}
```

The flow for the person at the counter:

1. Tap **Pair a cashier** here.
2. Start pairing on the cashier. This screen shows a 6-digit code.
3. Type the code on the cashier, then tap **Confirm** here.

Outside the 2-minute window, pairing requests are refused, so nobody can pair while the merchant isn't looking. `terminal.closePairing()` closes the window early.

`terminal.pairedCashiers()` lists the paired cashiers, and `terminal.forget(cashierDeviceId)` removes one. A forgotten cashier can no longer take payments until it pairs again.

4. Connect your card payment (PaymentAdapter)

`PaymentAdapter` is where the card is actually charged: your bank's payment app, Sunmi PayLib, or the built-in `SimulatedPayment` while you have none.

```
interface PaymentAdapter {
    val name: String
    val isSimulator: Boolean get() = false
    fun sale(request: SaleRequest): Flow<PaymentEvent>
    fun cancel(txnId: String): Boolean
}
```

The rules

- **`sale(request)`** returns a `Flow`. It emits `PaymentEvent.Stage(stage)` as the payment moves on, and **ends with exactly one `PaymentEvent.Finished(result)`**.
- **`cancel(txnId)`** returns `true` only if no card has been presented yet. The flow must then finish with outcome `CANCELLED`. Return `false` when it's too late.
- **Never block.** Both are called on the SDK's thread. Use suspending calls or `callbackFlow`.
- **Choose the outcome carefully.** The cashier treats every outcome except `APPROVED` as *not paid*:
- `APPROVED`: the bank approved, and money was taken.
- `DECLINED`: the bank said no.
- `CANCELLED`: cancelled before a card was read.
- `TIMEOUT`: no card was presented in time.
- `ERROR`: it failed, **and you are sure no money moved**.
- **Card data:** put only the masked card number (`476173******0119`) in `SaleResult`. Never the full number, track data, PIN or CVV.
- **Amount:** `result.amount` is what was actually charged, in minor units.

Example

The bank SDK below is made up. Replace it with the one you have.

```
class BankPayment(private val bank: BankSdk) : PaymentAdapter {
    override val name = "Bank app"

    override fun sale(request: SaleRequest): Flow<PaymentEvent> = callbackFlow {
        trySend(PaymentEvent.Stage(PaymentStage.WAITING_CARD))
        bank.startSale(request.amount, request.currency, request.txnId, object : BankSdk.Callback {
            override fun onCardRead() { trySend(PaymentEvent.Stage(PaymentStage.CARD_READ)) }
            override fun onPinRequested() { trySend(PaymentEvent.Stage(PaymentStage.PIN_ENTRY)) }
            override fun onSentToBank() { trySend(PaymentEvent.Stage(PaymentStage.AUTHORIZING)) }
            override fun onFinished(r: BankSdk.Result) {
                trySend(PaymentEvent.Finished(SaleResult(
                    txnId = request.txnId,
                    outcome = when {
                        r.approved -> Outcome.APPROVED
                        r.cancelled -> Outcome.CANCELLED
                        r.noCard -> Outcome.TIMEOUT
                        else -> Outcome.DECLINED
                    },
                    amount = r.amount,
                    currency = request.currency,
                    responseCode = r.responseCode,
                    message = r.message,
                    authCode = r.authCode,
                    rrn = r.rrn,
                    stan = r.stan,
                    maskedPan = r.maskedPan,
                    cardScheme = r.scheme,
                    terminalId = r.terminalId,
                    merchantId = r.merchantId,
                )))
                close()
            }
        })
        awaitClose { } // a started payment is never abandoned half-way
    }

    override fun cancel(txnId: String): Boolean = bank.cancelIfNoCard(txnId)
}
```

Try it with the simulator first

```
val simulator = SimulatedPayment()                // waits for presentCard(); 60 s card timeout
// or SimulatedPayment(autoOutcome = Outcome.APPROVED) to approve every sale by itself

// "Card: approve" / "Card: decline" buttons on the terminal screen while WaitingCard:
simulator.presentCard(approve = true)
```

`terminal.isSimulator` is `true` with the simulator, so you can show those buttons only then. No real card is charged.

5. What the SDK enforces for you

You don't need code for any of these:

- **One charge per `txnId`.** If a cashier sends the same `txnId` again, the stored result goes back and your adapter is not called.

- **One sale at a time.** Another sale meanwhile gets `BUSY`.
- **Results are delivered.** A result the cashier didn't confirm is re-sent when it reconnects (last 24 hours).
- **App restart mid-payment.** A payment that was running when the app died is marked unknown (`INTERRUPTED`), and the cashier is told to check the bank report.

`terminal.transactions()` lists the journal, newest first. `terminal.updates` emits every change, for example to show a list of today's payments.

Threads

What	Runs on
`start`, `openPairing`, `closePairing`, `cancelOnTerminal`, `dismissResult`, `forget`	Any thread; they return at once
`request.confirm()`, `request.reject()`	Any thread
`PaymentAdapter.sale` (the flow) and `cancel`	The SDK's thread: don't block
`StateFlow`s and `SharedFlow`s	Wherever you collect them

Known limitations

These hold for version 0.1.0:

- **An adapter can't report "outcome unknown".** If your bank app can end without knowing whether the card was charged (for example it crashes after contacting the bank), there is no event for that yet. Don't report `DECLINED` or `ERROR` in that case. Show the merchant a message to check the bank report.
- **Don't throw from the flow.** Catch errors inside your adapter and finish with an outcome. An exception that escapes the flow crashes the app. After the restart, the payment shows as interrupted.
- **One link per terminal**, USB or Bluetooth, not both at once.

Reference

```
class PosLinkTerminal(
    transport: Transport,
    identity: DeviceIdentity,
    pairings: PairingStore,
    journal: TransactionJournal,
    payment: PaymentAdapter,
    config: LinkConfig = LinkConfig(),
    info: Map<String, Any?> = emptyMap(),
    resultHoldMs: Long = 6000,                  // how long phase stays Result
    log: (String) -> Unit = {},
    // dispatcher, clock, scope: for tests
)

fun start()
suspend fun stop()

val status: StateFlow<LinkStatus>
val phase: StateFlow<TerminalPhase>
val pairingRequest: StateFlow<PairingRequest?> // .cashierName, .code, .confirm(), .reject()
val pairingResults: SharedFlow<PairingResult>
val updates: SharedFlow<TxnRecord>
val isSimulator: Boolean
val link: LinkSession                          // transportState, problem, peer, stats

fun openPairing(windowMs: Long = 120_000)
fun closePairing()
fun pairedCashiers(): List<Pairing>
fun forget(cashierDeviceId: String)
fun cancelOnTerminal()
fun dismissResult()
fun transactions(): List<TxnRecord>
```

CHAPTER 5

Testing without devices or cards

You can test most of your integration without a terminal, a cable or a card.

Level	Use	Needs
Unit tests on your computer	`MemoryTransportPair` + `SimulatedPayment`	Nothing
Two phones or emulators	`TcpServerTransport` / `TcpClientTransport` + `SimulatedPayment`	The same Wi-Fi
The real link	USB or Bluetooth + `SimulatedPayment`	Two devices; no card
End to end	USB or Bluetooth + your `PaymentAdapter`	Test cards from your bank

Unit tests (JVM)

`MemoryTransportPair` is a cable in memory: a cashier and a terminal run in one test, in milliseconds. Pre-share a key to skip pairing:

```
@Test fun approvedSale() = runBlocking {
    val wire = MemoryTransportPair()
    val cashierId = DeviceIdentity.random("Front desk")
    val terminalId = DeviceIdentity.random("Test terminal")

    // Skip pairing: give both sides the same key (tests only).
    val key = ByteArray(32) { 7 }
    val cashierPairings = InMemoryPairingStore().apply { put(Pairing(terminalId.idHex,
terminalId.name, key, 0)) }
    val terminalPairings = InMemoryPairingStore().apply { put(Pairing(cashierId.idHex,
cashierId.name, key, 0)) }

    val terminal = PosLinkTerminal(
        wire.b, terminalId, terminalPairings, InMemoryJournal(),
        SimulatedPayment(autoOutcome = Outcome.APPROVED, stepDelayMs = 10),
    )
    val cashier = PosLinkCashier(wire.a, cashierId, cashierPairings, InMemoryJournal())
    terminal.start()
    cashier.start()

    withTimeout(10_000) { cashier.status.first { it == LinkStatus.ONLINE } }
    val outcome = withTimeout(10_000) { cashier.sale(12500, "LYD") }

    assertTrue(outcome is SaleOutcome.Completed && outcome.result.approved)
    cashier.stop()
    terminal.stop()
}
```

This exact example is also a test in the SDK (`DocsExampleTest`), so it is known to pass.

Android's `org.json` doesn't work in local unit tests, so add the real one to your test dependencies:

```
dependencies {
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.json:json:20180813")
}
```

Breaking things on purpose

`MemoryTransport` (`wire.a`, `wire.b`) can misbehave on command, so you can test your app's handling of bad days:

Call	Simulates
`wire.disconnect()` / `wire.reconnect()`	Pulling the cable and plugging it back in
`wire.a.dropNext = 2`	The next 2 sends get lost (the SDK retries them)
`wire.a.chunkSize = 20`	Bytes arrive in small pieces, like Bluetooth
`wire.b.stall()` / `wire.b.unstall()`	The other side stops reading, like a busy USB accessory
`wire.a.interceptor = { bytes -> … }`	A device in the middle that changes or drops bytes

`SimulatedPayment` options:

Option	Default	Use
`autoOutcome`	`null` (wait for `presentCard`)	`Outcome.APPROVED` or `DECLINED` to finish by itself
`cardTimeoutMs`	60 000	Ends with `TIMEOUT` if no card is presented
`stepDelayMs`	900	Time per stage (card read, PIN, authorizing). Use 10 in tests.

Virtual time

For timeouts that take minutes (the 150-second no-update timeout, the 2-minute pairing window), run the SDK on `kotlinx-coroutines-test` virtual time. Pass the test's dispatcher, clock and scope:

```
@Test fun noUpdateBecomesUnknown() = runTest {
    val dispatcher = coroutineContext[ContinuationInterceptor] as CoroutineDispatcher
    val clock = { testScheduler.currentTime }
    val cashier = PosLinkCashier(
        wire.a, cashierId, cashierPairings, InMemoryJournal(),
        dispatcher = dispatcher, clock = clock, scope = backgroundScope,
    )
    // same for the terminal; then advanceTimeBy(…) instead of waiting
}
```

The SDK's own tests in `android/poslink/src/test` use this pattern, including lost frames, a pulled cable and tampering.

Two phones on the same Wi-Fi

```
// Terminal phone
PosLinkTerminal(TcpServerTransport(8765), /* … */, payment = SimulatedPayment())

// Cashier phone (use the terminal phone's IP address)
PosLinkCashier(TcpClientTransport("192.168.1.50", 8765), /* … */)
```

Add `<uses-permission android:name="android.permission.INTERNET" />` to both apps.

On two emulators on one computer, the cashier connects to `10.0.2.2`, and you forward the port to the terminal emulator with `adb -s <terminal-emulator> forward tcp:8765 tcp:8765`.

The sample app

`android/sample` is a small app that plays either role over USB, Bluetooth or Wi-Fi, with the simulator on the terminal side. Install it on two devices to see a working setup before writing your own:

```
cd android && ./gradlew :sample:installDebug
```

Its source (`MainActivity.kt`) is a compact, complete example of both sides.

CHAPTER 6

Payment safety checklist

Go through this before real money goes through your integration. The SDK prevents double charges and fake results on the link. Your app still has to use the results correctly.

The three rules

1. **One order, one `txnId`.** Create it before the first attempt, save it with the order, and reuse it for every retry of that order. The terminal charges a `txnId` at most once. A new `txnId` is a new charge.
2. **Only `Completed` with `result.approved` means paid.** Everything else means not paid, except `Unknown`.
3. **`Unknown` is not "declined".** The card may have been charged. Don't charge again with a new `txnId` until you know. The SDK finds out by itself when the link is back.

Cashier app

- [ ] Each order stores its `txnId`, written **before** `sale()` is called.
- [ ] Retrying a payment for the same order passes the same `txnId`.
- [ ] Only `SaleOutcome.Completed` with `result.approved == true` marks the order paid.
- [ ] `SaleOutcome.Unknown` puts the order in a visible "needs checking" state, never "failed".
- [ ] `cashier.updates` is collected app-wide, so late results (after a dropped link) update the order. See the cashier guide.
- [ ] Orders in "needs checking" have a **Check payment** action (`cashier.status(txnId)`).
- [ ] Amounts are in minor units, with the right number of decimals for the currency (LYD: 3, so 12.500 LYD = `12500`).
- [ ] The receipt uses the values from `SaleResult`: amount, auth code, RRN, masked card number.
- [ ] Production uses `FileJournal` and `KeystorePairingStore`, never the in-memory versions.
- [ ] Pairing asks the user to **type** the code shown on the terminal.

Terminal app

- [ ] Your `PaymentAdapter` reports `APPROVED` only when the bank approved.
- [ ] It reports `ERROR` or `DECLINED` only when no money moved.
- [ ] It never throws out of its flow (see known limitations).
- [ ] Only the masked card number leaves the adapter. Never the full number, track data, PIN or CVV.
- [ ] Production uses `FileJournal` and `KeystorePairingStore`.
- [ ] **Pair a cashier** sits in a merchant-only screen (behind a PIN, for example).
- [ ] `SimulatedPayment` is not in the production build.
- [ ] On Sunmi production devices, the app is installed through the SUNMI App Store. Production devices don't accept other installs.

Failure drills

Run each drill on the real devices, with the real link (USB or Bluetooth), before going live. Use test cards, or the simulator first.

#	Do this	Expected
1	Pull the cable (or turn Bluetooth off) while the terminal says "Present card", then reconnect	The sale goes on, or ends with a clear outcome. No double charge.
2	Pull the cable during "Authorizing", reconnect after the terminal shows the result	The cashier gets the real result after reconnecting (`updates`, or the running `sale()`)
3	Force-stop the cashier app mid-payment, then reopen it	The sale is listed as unknown, then resolved once the link is back
4	Force-stop the terminal app mid-payment, then reopen it	The cashier gets `Unknown` (interrupted). The bank report shows the truth.
5	Start a second sale while one is running	`Refused` with code `BUSY`
6	After an approved sale, call `sale()` again with the same `txnId`	The same approved result comes back. The bank report shows one charge.
7	Cancel before presenting a card, then again after presenting one	First: `CANCELLED`. Second: the payment finishes normally.
8	Reinstall the terminal app	The cashier shows `UNPAIRED` and a problem message. Pair again, then sales work.
9	Pair with a wrong code typed on the cashier	Pairing fails. No pairing is saved on either side.

What the link protects against

- **A device in the middle** (a USB hub, a cable with a chip, a Bluetooth relay) can't fake a result, change an amount or replay an old "approved". Every message is signed with the key from pairing and numbered, and anything else is dropped.
- **A stranger can't pair** with the terminal unless someone at the terminal opens pairing and confirms the code.
- **Copying the app's files** to another device doesn't copy the pairing. The key that unlocks it stays in that device's Android Keystore.

It does not protect against someone who controls an unlocked, already-paired cashier. That person can start sales like any cashier user, but the customer still has to present a card. Keep the cashier app behind your normal staff login.

CHAPTER 7

Troubleshooting

First, turn on the SDK's log and read `link.transportState` and `link.problem`. Most problems explain themselves there.

```
PosLinkCashier(/* … */, log = { Log.d("PosLink", it) }) // same for PosLinkTerminal
```

```
adb logcat -s PosLink
```

The status stays OFFLINE

Bluetooth

Check	Fix
`transportState` is `PERMISSION`	Ask for `requiredPermissions()` (setup). The link starts on its own once they are granted.
`ERROR: Bluetooth is off…`	Turn Bluetooth on. The link notices on its own.
The two devices aren't paired in Android settings	Pair them in **Settings → Bluetooth** (setup).
The cashier has the wrong address	Pick the terminal again from `PosLinkBluetooth.bondedDevices()`.
The terminal app isn't running, or hasn't called `start()`	Open it. Its status should say it's waiting for the cashier.
The terminal runs the old prototype app (protocol v1)	v1 and v2 listen on different Bluetooth service ids, so they never meet. Install the v2 app.

The cashier retries every 2 to 15 seconds, so after a fix it can take up to 15 seconds to connect.

USB

Check	Fix
`transportState` is `PERMISSION: Allow USB access on the screen`	Tap **OK** in the system dialog. Tick "Always open" so it isn't asked again.
`ERROR: USB permission denied…`	Unplug and replug the cable to get the dialog again.
`WAITING: Looking for the terminal…` and nothing happens	Use a **data** cable: many cables only charge. Then check that the terminal's USB id is in your device filter (setup).
`… does not answer the Android accessory (AOA) handshake`	That device can't act as a USB accessory. Plug the other end in, or use another device.
The terminal shows "No installed app works with this USB accessory"	The terminal app isn't installed, or its manifest lacks the accessory filter.

Check	Fix
`ERROR: Unknown USB accessory: …`	The cashier app uses another identity: an old v1 app, or a different product.

The cashier is always the USB **host**: it needs USB host support (USB-C or OTG). The terminal is the accessory, and needs no developer options or USB debugging.

Stuck at HANDSHAKING

The transport is connected, but the other app doesn't answer. Look at `link.problem`:

Problem	Fix
`Both devices are set to "cashier"` (or `"terminal"`)	One device must be the cashier and the other the terminal.
`The other device runs POS Link protocol v1`	Update that app to the v2 SDK.
`The other device speaks protocol vN…`	Update both apps to the same SDK major version.
No problem shown	The other app is busy or frozen. Restart it.

UNPAIRED, although it worked before

The problem says something like `The terminal no longer knows this device`. One of the two apps was reinstalled or had its data cleared, so its key is gone. Pair again. You don't have to remove the old pairing first.

Stuck at AUTHENTICATING, or "failed the security check"

The two sides have different keys for each other. This happens, for example, when the cashier was paired again with a reinstalled terminal while another cashier kept an old pairing. Call `forget()` on both sides for that peer, then pair again.

If it happens with no reinstall at all, something may be changing bytes on the link (a bad cable or hub). After 5 failed checks the SDK resets the connection.

Pairing fails

`PairingResult.Failed.reason`	Fix
`NOT_PAIRING`	Tap **Pair a cashier** on the terminal first. Pairing stays open for 2 minutes.
`BAD_CONFIRM`	The codes didn't match. Try again, and type the code exactly as the terminal shows it.
`REJECTED`, `TIMEOUT`	Someone tapped Reject, or nobody finished in 2 minutes. Try again.
`NOT_CONNECTED`	Pairing needs a connected terminal: status must be at least `UNPAIRED`.

Sales

What you see	Why	Fix
`Refused`: "Terminal not connected"	Status isn't `ONLINE`	Check the status before charging. Enable the Pay button only when `ONLINE`.
`Refused` with code `BUSY`	A payment is already running on the terminal, or on this cashier	Wait, or cancel on the terminal
`Refused` with code `BAD_REQUEST`	Amount ≤ 0, currency not 3 letters, or empty `txnId`	Fix the request
`Unknown`	The link dropped mid-payment, or the terminal went silent for 150 s	See the cashier guide. Never treat it as declined.
`NOT_RESPONDING` during a sale	The terminal app froze, or the link is flaky	The sale keeps waiting. If the link drops, it becomes `Unknown` and is resolved later.

Bluetooth LE

POS Link doesn't use Bluetooth LE. Sunmi terminals in payment-security (PCI) mode stop Android's LE service as soon as Bluetooth turns on, so LE can't work there. Bluetooth Classic works.

Installing on Sunmi devices

- **Development devices** take normal `adb install`.
- **Production devices** install only from the SUNMI App Store. Publish the terminal app there.
- On Sunmi P3 firmware 3.0.x, the P3's USB id is `0E8D:201C`, with developer options on or off. If your firmware shows a different id, add it to your device filter.